



Perfect Demo Makes Poor Teacher: Learning Robust Alignment from Critical Motion Segments

Mingyu Liu^{1,2*} Zeju Li^{1*} Jiuhe Shu¹

Hanqing Wang³ Yuhao Chao^{2,4} Hao Chen¹ Chunhua Shen^{1†}

¹Zhejiang University ²Shanghai Innovation Institute

³Hong Kong University of Science and Technology (GZ) ⁴Nanjing University

aim-uofa.github.io/STAIR

A Training Details

This appendix provides comprehensive training details for reproducing the STAIR model and experiments described in the main paper.

Table 1: **Reproducibility summary.** The real-robot experiments use matched trajectory counts across data conditions and the same evaluation protocol across methods.

Item	Setting
Robot platform	Dual-arm PiperX robot from AgileX Robotics
Observations	One head camera and two wrist cameras; RGB resized/cropped as described below
Tasks	Six manipulation tasks grouped into Easy, Middle, and Hard alignment levels
Demonstrations	100 trajectories per task for each data condition unless otherwise stated
Data conditions	Fluent demonstrations, deliberate demonstrations, and critical-segment resampling of fluent demonstrations
Real-robot evaluation	15 closed-loop trials per method and task, with identical reset and success criteria across methods

A.1 Model Architecture

Vision-Language Model (VLM) Backbone. We use the Prismatic/OpenVLA backbone [1] with the prism-dinosiglip-224px+7b configuration:

- **Vision encoder:** a DinoSigLIP backbone that concatenates patch features from DINOv2 ViT-L/14 [2] and SigLIP So400M at 224×224 resolution. The image resize strategy is resize-naive
- **Language model:** LLaMA-2-7B, both the vision backbone and the LLM backbone are full fine-tuned
- **Visual projector:** the Prismatic fused-GELU MLP projector that maps the concatenated visual features into the 4096-dim LLM embedding space

Action Expert. The action expert is a flow-matching DiT-B action head that generates action chunks:

- Hidden dimension: 768
- Transformer layers: 12
- Attention heads: 12
- Action chunk length: $H = 16$ steps (covering ~ 1 s of future actions at 15 Hz control frequency)

*Equal contribution. †Corresponding author.

- Flow matching: training samples flow timesteps from Beta(1.5, 1.0) and repeats each batch item four times by default; inference uses 10 Euler steps unless otherwise specified
- Conditional input: a 4096-dim VLM cognition token is prepended to the action sequence; dynamic tokens enter every DiT block through multi-head cross-attention

Dynamic Feature Pathway. The dynamic alignment feature extractor consists of:

- **WanVAE encoder** E_{wan} : a frozen Wan video VAE. We use only its encoder branch, which receives a 5-frame RGB clip at 256×256 resolution and maps the local observation neighbourhood to a spatiotemporal video latent
- **3D latent projection**: a trainable 3D convolution with kernel and stride (1, 2, 2) patchifies the WanVAE latent and maps each latent patch to a 4096-dimensional video token
- **Q-Former projector** P_ψ : 2 learnable query tokens attend to the video-token sequence through 6 cross-attention layers with 8 attention heads, producing compact dynamic tokens $\mathbf{f}_t^{\text{dyn}}$ with shape $[B, 2, 4096]$
- **MLP adapter** MLP_η : a trainable GELU MLP with the shape $8192 \rightarrow 8192 \rightarrow 8192$ maps the flattened dynamic tokens into the action-conditioning space and reshapes them back to 2 tokens of dimension 4096
- **Condition interface**: the adapted dynamic tokens have shape $[B, 2, 4096]$ and are passed to the DiT per-token embedder before cross-attention

Query Module (Stage II). The QFormer query module that distills dynamic features from the VLM hidden states:

- 2 learnable query tokens of dimension 4096
- 4 transformer layers with 8 attention heads
- Cross-attention to the last four language hidden states after removing vision patches
- Output projection: $\text{Linear}(4096 \rightarrow 4096)$, producing the same $[B, 2, 4096]$ condition shape as the frozen dynamic feature pathway

A.2 Hyperparameters

Table 2 summarizes the default hyperparameters used by the STAIR VLA training scripts. The STAIR dynamic-feature pathway is trained before these two VLA stages and is frozen when supervising the policy. Stage II loads the Stage I checkpoint and changes which STAIR-specific modules are trainable.

A.3 Data Collection and Preprocessing

Collection protocol. For each task, fluent and deliberate demonstrations are matched in trajectory count and collected on the same robot setup. Fluent demonstrations ask the operator to complete the task naturally and efficiently. Deliberate demonstrations keep the same task objective but change the teaching style near the final contact or placement phase: the operator slows down, exposes small corrective motions, and occasionally enters mild off-axis states before recovering. The intent is not to make the whole trajectory slower, but to allocate more observable behavior to the high-sensitivity alignment phase.

Demonstration collection. Each task is demonstrated on a dual-arm PiperX robot (AgileX) with one head camera (110° FoV, 30 FPS, 640×360) and two wrist cameras (160° FoV, 30 FPS, 640×360). We collect 100 trajectories per task under each data condition:

- **Fluent:** Operators execute the task at natural speed with efficient, straight-line motions.
- **Deliberate:** Operators slow down near contact, add small corrective motions, and occasionally approach with mild offsets before recovering.

Task success criteria. We use binary success labels at the end of each closed-loop rollout. The criteria are fixed before evaluation and applied identically to all methods:

- **Stack Bowls:** the target bowl is placed on the support bowl and remains stable after release.

Table 2: Training hyperparameters for STAIR.

Hyperparameter	Stage I	Stage II
Data mixture	bridge_rt_1	
Training strategy	FSDP full-shard	
#GPUs	8	
Per-GPU batch size	32	
Global batch size	256	
Gradient accumulation steps	1	
Trainable VLM modules	Vision backbone, projector, and LLM	
Trainable STAIR modules	MLP adapter, action expert cross-attention/action head	Query module and action expert
Frozen STAIR modules	WanVAE encoder and Q-Former projector P_ψ	WanVAE encoder, P_ψ , and MLP adapter in the STAIR dynamic-feature path
Optimizer	AdamW over trainable parameters	
Learning rate	2×10^{-5}	
LR schedule	Constant	
Warmup ratio	0.0	
Optimization budget	10,000 steps	5,000 steps
Weight decay	0.0	
$(\beta_1, \beta_2, \epsilon)$	(0.9, 0.999, 10^{-8})	
Gradient clipping	1.0	
λ_{action} (flow-matching loss)	1.0	
λ_{dyn} (cosine distillation loss)	N/A	0.5
Repeated diffusion samples	4	
Mixed precision	BF16 parameters with FP32 gradient reduction	

- **Stack Lego Blocks:** the top block is placed on the target block without falling or sliding off after release.
- **Insert Flowers:** the flower stem is inserted into the receptacle and remains upright after release.
- **Insert Pen Cap:** the cap is aligned and inserted onto the pen without jamming or dropping.
- **Put Coin into Safe:** the coin enters the slot or safe opening completely and is not left on the surface.
- **Open the Lock:** the lock is manipulated to the open state rather than merely contacted or displaced.

Image preprocessing.

1. Resize to 320×320 (center crop from 640×360).
2. Random crop to 224×224 during training; center crop at test time.
3. Color jitter (brightness ± 0.2 , contrast ± 0.2 , saturation ± 0.2 , hue ± 0.1) with probability 0.5.
4. Normalize with ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225).

Action representation. Actions are 7-DoF: 6-DoF end-effector delta pose ($\Delta x, \Delta y, \Delta z, \Delta \text{roll}, \Delta \text{pitch}, \Delta \text{yaw}$) plus 1-DoF gripper open/close binary command. We normalize each dimension to $[-1, 1]$ using per-task min-max statistics computed from the training set.

Dynamic feature inputs. For each action chunk at time t , we extract a 5-frame video clip from the head camera at temporal offsets $\Delta = \{-2, -1, 0, +1, +2\}$ relative to t . The clip is resized to 256×256 and normalized to $[-1, 1]$ before feeding into the WanVAE encoder. This neighbourhood is used only to construct training-time teacher targets; at deployment, the policy receives only the current observation and language instruction.

A.4 Training Protocol

STAIR dynamic-feature pathway training. The STAIR dynamic-feature pathway is trained before the VLA policy and provides the motion-aware targets used by STAIR. Its training uses short robot video clips. Given a local clip $\mathcal{N}_t$, the frozen WanVAE encoder produces a video latent $E_{\text{wan}}(\mathcal{N}_t)$. A trainable projector P_ψ , consisting of the 3D latent projection and Q-Former compression described in Sec. A.1, maps this latent to compact dynamic tokens:

$$\mathbf{f}_t^{\text{dyn}} = P_\psi(E_{\text{wan}}(\mathcal{N}_t)). \quad (1)$$

These tokens condition a Wan video flow model. During training, the video latent is mixed with Gaussian noise at a sampled flow timestep, and the video transformer predicts the corresponding velocity target:

$$\mathcal{L}_{\text{VM}} = \mathbb{E}_{s, \epsilon} \left[\left\| v_{\text{wan}} \left((1-s)E_{\text{wan}}(\mathcal{N}_t) + s\epsilon, s, \mathbf{f}_t^{\text{dyn}} \right) - (\epsilon - E_{\text{wan}}(\mathcal{N}_t)) \right\|_2^2 \right]. \quad (2)$$

The WanVAE remains frozen. This pathway training updates the video transformer and the encoder-side projector P_ψ ; in our BridgeV2 configuration, the Wan transformer is adapted with LoRA while the 3D latent projection and Q-Former projector are trained directly. After this training, the VLA uses the frozen E_{wan} and P_ψ path to produce $\mathbf{f}_t^{\text{dyn}}$. The video denoising transformer supplies the training signal for the STAIR dynamic-feature pathway, but it is not called by the VLA policy during rollout.

STAIR VLA Two-Stage Training. The STAIR VLA policy is trained after the dynamic-feature pathway is available:

1. **Stage I:** start from the pretrained VLA backbone and attach the MLP adapter and flow-matching action expert. The frozen STAIR dynamic-feature pathway produces $\mathbf{f}_t^{\text{dyn}}$ from the dynamic video clip, and the adapter maps it to the action-conditioning space, $\mathbf{c}_t^{\text{dyn}} = \text{MLP}_\eta(\mathbf{f}_t^{\text{dyn}})$. We train the adapter, the action expert, and the selected VLA modules with the action flow-matching objective $\mathcal{L}_I$. Gradients from the action loss do not update E_{wan} or P_ψ .
2. **Stage II:** initialize from the Stage I policy, keep the STAIR dynamic-feature pathway and the MLP adapter fixed, and add a QFormer query module on top of the VLM hidden states. The action expert continues to train with the frozen condition $\mathbf{c}_t^{\text{dyn}}$, while the query module predicts $\hat{\mathbf{c}}_t^{\text{dyn}}$ from the current observation and instruction. We supervise this prediction with a cosine distance to the frozen STAIR condition:

$$\mathcal{L}_{\text{II}} = \mathcal{L}_{\text{action}}(z_t, \mathbf{c}_t^{\text{dyn}}) + \lambda_{\text{dyn}} \left(1 - \mathcal{S}(\hat{\mathbf{c}}_t^{\text{dyn}}, \mathbf{c}_t^{\text{dyn}}) \right).$$

3. **Inference:** the training-time dynamic video path is removed. The query module predicts the dynamic condition from the current RGB observation and instruction, and the action expert generates the action chunk from the VLM state and the predicted condition.

Validation. For real-robot training, we hold out 10% of each task’s demonstrations with a stratified split over task and data condition. Validation is performed every 500 optimization steps, and we select the checkpoint with the lowest validation objective for evaluation. The same selection rule is used for the baseline and STAIR variants.

A.5 Critical-Segment Resampling

For the resampling baseline, we annotate task-critical moments $\mathcal{K}$ per trajectory. These moments include gripper close/open events, first sustained contact, insertion start, placement, or lock actuation, depending on the task. We then define a critical window

$$\mathcal{W} = \{t : \min_{k \in \mathcal{K}} |t - k| \leq r\},$$

with radius $r = 10$ frames. Frames inside $\mathcal{W}$ are upweighted by $\lambda = 5$ in the loss (Eq. 2 in the main paper). In the alignment-coverage analysis, this raises the effective alignment budget from

approximately 22 raw alignment frames per fluent trajectory to approximately 88 effective frames per trajectory. Training hyperparameters are otherwise identical to the corresponding fluent-data baseline, so the comparison isolates critical-segment weighting without changing the recorded state distribution.

A.6 Hardware and Runtime

- **Training GPUs:** $8 \times$ NVIDIA A100 80 GB for the real-robot VLA experiments.
- **Stage I wall-clock:** ~ 12 h (10,000 steps).
- **Stage II wall-clock:** ~ 6 h (5,000 steps).
- **Total per task configuration:** ~ 18 h.
- **SimplerEnv pre-training:** ~ 48 h on $8 \times$ A100.
- **Framework:** PyTorch 2.1, CUDA 12.1, PyTorch DDP, BF16 mixed precision.

B Additional Experiments

Unless otherwise noted, the ablations below follow the same real-robot training and evaluation protocol as the main experiments: the same six tasks, the same demonstration counts, and 15 closed-loop trials per task. We report success rates averaged within each difficulty group, and the Overall column is the mean across Easy, Middle, and Hard. These ablations are intended to validate implementation choices rather than introduce additional task settings.

B.1 SimplerEnv Evaluation

We evaluate general manipulation in SimplerEnv, following a standard evaluation protocol. SimplerEnv is a real-to-sim proxy for policies trained on Fractal and Bridge-style data, and it exposes two robot embodiments: the Google Robot and the WidowX Bridge setup. All runs are closed-loop. At every control step, the STAIR VLA receives the current RGB observation and language instruction only; the training-time dynamic video path is not available.

Evaluation suites. The default setting `SUITES=all` expands to three groups:

- **Google Robot visual matching:** Pick Coke Can, Move Near, and Drawer tasks with RGB overlays and scene settings chosen to match the corresponding real evaluations.
- **Google Robot variant aggregation:** the same task families under stronger distribution shift, including background changes, lighting changes, table-texture changes, distractors, and station variants.
- **WidowX Bridge:** four Bridge pick-and-place or stacking tasks: Stack Green Cube on Yellow Cube, Put Carrot on Plate, Put Spoon on Towel, and Put Eggplant in Basket.

Rollout protocol. The Google Robot suites use the standard Google Robot controller at 3 Hz with a 513 Hz simulation frequency. Pick Coke Can and Move Near episodes are capped at 80 steps, while Drawer episodes are capped at 113 steps. The WidowX Bridge suite uses a 5 Hz controller with a 500 Hz simulation frequency and a 120-step horizon. The script evaluates all 313 generated task specifications by default, covering 2,385 closed-loop rollouts in total: 756 rollouts for Google visual matching, 1,533 for Google variant aggregation, and 96 for WidowX Bridge.

Stage-II-only baseline. We also evaluate a Stage II-only baseline to isolate the effect of using the full two-stage STAIR training recipe. This baseline trains the VLA action path directly in the second stage, without first learning the STAIR dynamic feature interface. Tables 3 and 4 report the resulting task success rates on the same standard SimplerEnv suites.

Table 3: Stage-II-only baseline on Google Robot tasks. Values are task success rates.

Model	Google Visual Matching				Google Variant Aggregation				Overall Avg.
	Pick Coke	Move Near	Drawer	Avg.	Pick Coke	Move Near	Drawer	Avg.	
Stage-II only	80.7%	62.1%	63.9%	68.9%	82.2%	64.8%	30.2%	59.1%	64.0%

Table 4: Stage-II-only baseline on WidowX Bridge tasks. Values are final task success rates.

Model	Put Spoon on Towel	Stack Green on Yellow	Put Carrot on Plate	Put Eggplant in Basket	Success Avg.
Stage-II only	37.5%	8.3%	33.3%	100.0%	44.8%

B.2 Video-Token Budget Before Q-Former Compression

Table 5 studies the effect of the number of video tokens retained after 3D latent projection and before Q-Former compression. The final condition shape remains two 4096-dimensional tokens, matching the action expert interface; only the intermediate video-token budget is varied. Too few tokens under-represent local motion around contact, while too many tokens add redundant detail and slightly reduce performance on harder tasks.

B.3 Temporal Window Size

Table 6 studies the number of frames used for STAIR dynamic feature extraction. In all cases, the video neighbourhood is used only for training-time supervision; deployment still uses the current RGB observation and instruction only. A 5-frame neighbourhood provides the best trade-off: 3 frames miss part of the corrective motion, while 7 frames add longer-horizon variation that is less tightly aligned with the action chunk.

Table 5: **Video-token budget** before Q-Former compression. Success rate (%).

Budget	Easy	Middle	Hard	Overall
64	90.0	60.0	20.0	56.7
128 (default)	93.3	70.0	36.7	66.7
256	93.3	66.7	33.3	64.4

Table 6: **Temporal window size** for dynamic features. Success rate (%).

Window	Easy	Middle	Hard	Overall
3 frames	90.0	63.3	26.7	60.0
5 frames (default)	93.3	70.0	36.7	66.7
7 frames	93.3	66.7	33.3	64.4

B.4 Flow Matching Inference Steps

Table 7 studies the trade-off between inference speed and quality. Increasing from 5 to 10 flow-matching inference steps improves alignment-sensitive tasks, while 20 steps gives no additional success-rate gain but doubles action-head latency.

B.5 Distillation Loss Weight

Table 8 studies the effect of λ_{dyn} in Stage II. A small distillation weight underuses the dynamic supervision, while a larger weight over-emphasizes matching the dynamic condition at the expense of the action objective. We therefore use $\lambda_{\text{dyn}} = 0.5$ by default.

Table 7: **Flow-matching inference steps**. Success rate (%) and action-head latency.

Steps	Easy	Middle	Hard	Overall	Lat.
5	90.0	63.3	26.7	60.0	12
10 (default)	93.3	70.0	36.7	66.7	24
20	93.3	70.0	36.7	66.7	48

Table 8: **Distillation loss weight** λ_{dyn} . Success rate (%).

λ_{dyn}	Easy	Middle	Hard	Overall
0.1	90.0	60.0	23.3	57.8
0.5 (default)	93.3	70.0	36.7	66.7
1.0	93.3	66.7	33.3	64.4

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [2] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.